

Eternal Rover Mission Control // Inter IIT Tech Meet 14.0

This dashboard serves as the central command unit for the Autonomous Rover. It visualizes real-time telemetry, manages mission rounds, and maps the warehouse inventory status (Racks A-F) as data is received from the Jetson Nano.

Project Structure

- `app.py` : The main Flask server. Handles API requests and manages data storage.
- `restart_run.py` : A utility script that **deletes old data** (CSVs) and starts a fresh server instance.
- `test.py` : A simulation script to test the dashboard without the physical rover.
- `templates/` : Contains the HTML frontend (`dashboard.html` , `login.html`).
- `*.csv` : Data files generated automatically (`rover_data.csv` , `inventory.csv`).

How to Run

Option 1: Fresh Start (Recommended for Demos)

Use this command to **wipe all previous history** and start from Round 1.

```
python restart_run.py
```

Option 2: Resume Mission (Persistence Mode)

Use this command to keep existing data and **resume** from the last known round.

```
python app.py
```

Accessing the Dashboard

1. Open your browser and navigate to: `http://localhost:5000`
2. **Login Credentials:**
 - Username: `admin`
 - Password: `eternal2025`

Jetson Nano Integration Guide

The rover communicates with the dashboard via HTTP POST requests.

Endpoint: `http://<DASHBOARD_IP>:5000/api/update`

Method: POST

Content-Type: application/json

1. JSON Payload Format

The server expects a flat JSON object. All fields are optional, but providing more data populates more UI elements.

Field	Type	Description	Example
battery_level	float	Battery percentage (0-100).	88.5
cpu_temp	float	CPU Temperature in Celsius.	45.2
status	string	Short status message for the badge.	"Scanning"
current_task	string	Description of current action.	"Moving to Rack B"
coordinates	string	X, Y position string.	"12.5, 4.0"
shelf_id	string	Required for Inventory Map. Format: Rack - Level .	"A-05"
shelf_status	string	Status of the shelf.	"Empty" or "Full"
new_round	bool	Set to true to force a new round to start.	true

2. Python Example (Requests Library)

Use this snippet on the Jetson Nano:

```
import requests

SERVER_URL = "[http://192.168.1.100:5000/api/update](http://192.168.1.100:5000/api/updc

# 1. Telemetry Only Update
telemetry_payload = {
    "battery_level": 75.0,
    "cpu_temp": 55.0,
    "status": "Navigating",
    "coordinates": "1.2, 3.4"
}
requests.post(SERVER_URL, json=telemetry_payload)

# 2. Shelf Scan Update (Updates the Visual Grid)
scan_payload = {
    "status": "Scanning",
    "shelf_id": "C-03",          # Valid: A-01 to F-05
```

```
        "shelf_status": "Empty" # Turns box RED on dashboard
    }
    requests.post(SERVER_URL, json=scan_payload)

# 3. Start New Round (Resets Map)
requests.post(SERVER_URL, json={"new_round": True})
```

Warehouse Configuration

The system is hardcoded for the Inter IIT Blueprint:

- **6 Racks:** A, B, C, D, E, F
- **5 Levels:** O1 to O5
- **Total Slots:** 30

To change this, edit the `WAREHOUSE_MAPPING` dictionary in `app.py` .